Smart office management

Project report submitted in partial fulfillment
of the requirements for the degree of

Bachelor of Technology
in
Communication and Computer Engineering

by

Yuvraj Sehgal - 23UCC621

Under Guidance of
Dr. Saurabh Kumar



Department of Communication and Computer Engineering
The LNM Institute of Information Technology, Jaipur

April 2026

©The LNM INSTITUTE OF INFORMATION TECHNOLOGY Jaipur-2022.
All rights reserved.

The LNM Institute of Information Technology
Jaipur, India

CERTIFICATE

This is to certify that the project entitled “Smart office management” , submitted by Yuvraj Sehgal (23UCC621) in partial fulfillment of the requirement of degree in Bachelor of Technology (B. Tech), is a bonafide record of work carried out by them at the Department of Communication and Computer Engineering, The LNM Institute of Information Technology, Jaipur, (Rajasthan) India, during the academic session 2025-26 under my supervision and guidance and the same has not been submitted elsewhere for award of any other degree. In my/our opinion, this report is of standard required for the award of the degree of Bachelor of Technology (B. Tech).

Date

Adviser: Dr. Saurabh Kumar

Acknowledgments

I would like to express my sincere gratitude to my project supervisor, Dr. Saurabh Kumar for his continuous guidance, valuable suggestions, and support throughout the development of this project.

I am also thankful to LNMIIT for for providing the necessary resources and academic environment required for the successful completion of this project.

Finally, I am deeply grateful to my family for their constant support and encouragement.

Abstract

This project presents the design and implementation of a real-time facial recognition system for smart office management, deployed entirely on an edge device — a Raspberry Pi 5 with 8 GB RAM. The system eliminates dependence on cloud infrastructure, ensuring low latency and stronger privacy guarantees by keeping all computation and data local to the device. The pipeline is structured into distinct stages: live video is captured via a USB camera and each frame is processed by a Haar Cascade classifier for rapid face detection. Detected faces are assigned persistent identities across frames using an Intersection-over-Union (IOU) based tracker, which prevents redundant recognition computations. For each newly detected face, a cropped image is forwarded to the DeepFace framework using the GhostFaceNet model to generate a high-dimensional facial embedding vector. This embedding is compared against a pre-built database of known individuals using cosine similarity, and the closest match above a defined threshold is displayed as an identity label on the video feed. To maintain real-time frame rates on constrained hardware, recognition tasks are offloaded to background threads managed by a semaphore, decoupling the display pipeline from the computationally intensive embedding generation. The system achieves reliable frontal face recognition under standard lighting conditions, even at distances up to eight feet, and supports simultaneous recognition of multiple individuals. Performance evaluation demonstrates a one-time initialization delay of approximately ten seconds due to model loading, after which the system operates in near real-time. Known limitations include reduced accuracy under poor lighting, extreme head orientations, and rapid facial movement — constraints typical of resource-limited edge deployments. Future work includes integration of a Time-of-Flight (ToF) sensor with an ESP32-C3 microcontroller to enable event-driven activation, reducing idle power consumption, along with automatic database enrollment of unknown individuals and room automation based on occupancy data. The project demonstrates that deep learning-based facial recognition is viable on low-cost edge hardware, providing a practical foundation for intelligent, privacy-preserving office automation.

Contents

List of Figures viii

List of Tables ix

1 Introduction 1
- 1.1 The Area of Work . . . 1
- 1.2 Problem Addressed . . . 1
- 1.3 Existing Systems . . . 1
  - 1.3.1 Facial biometrics . . . 1
  - 1.3.2 Sensor controlled lights . . . 2
  - 1.3.3 Cloud-based vision . . . 2

2 Literature Review 3
- 2.1 Introduction . . . 3
- 2.2 Evolution of Face Detection and Recognition . . . 3
- 2.3 Lightweight Models for Edge Deployment . . . 4
- 2.4 Face Alignment and Multi-Task Detection . . . 5
- 2.5 Object Tracking and Identity Persistence . . . 5
- 2.6 Smart Office and Occupancy Detection Systems . . . 5
- 2.7 Concurrency and Real-Time Constraints on Edge Devices . . . 6
- 2.8 Summary and Research Gap . . . 6

3 Proposed Work 8
- 3.1 System overview . . . 8
- 3.2 Components used . . . 8
- 3.3 System architecture . . . 8
- 3.4 System Working . . . 10
- 3.5 Database creation . . . 10

4 Simulation and Results 12
- 4.1 Setup . . . 12
- 4.2 Performance evaluation . . . 12
  - 4.2.0.1 Initial response time . . . 12
  - 4.2.0.2 Real time performance . . . 12
  - 4.2.0.3 Recognition Accuracy(Qualitative) . . . 13
  - 4.2.0.4 Stability . . . 13
- 4.3 Limitations . . . 13

5 Conclusions and Future Work **16**

Bibliography **16**

List of Figures

3.1 Use case diagram 8
3.2 System Architecture of Proposed Model 9
3.3 Sequence diagram 10

4.1 Single face 14
4.2 Multiple Faces 15

List of Tables

4.1 Performance Summary of the Proposed System 13

Chapter 1

Introduction

Currently, not much work has been done to automate personal offices. The existing products focus on smart homes due to their marketability. A personal office can be smart in many ways and the main one which will be focused in this project will be real time, low latency, automated facial recognition to manage records without having to manually lineup in front of a device. This will also include occupancy detection of the room which can further be used to make decisions like light control, meeting mode and much more.

1.1 The Area of Work

This project falls under the field of Edge AI and IoT, focused on running high performance deep learning models on a resource constrained hardware. Unlike standard cloud-based AI, main focus is on edge computing *i.e.* on device processing for lower latency and user privacy.

1.2 Problem Addressed

There is not much work done on occupancy detection and personnel identification on edge devices. The standard ”Smart office” solutions rely on just motion sensors to automate simple fuctions like light control. Furthermore, the current occupancy detection systems are based on cloud computing which has privacy risks.

1.3 Existing Systems

1.3.1 Facial biometrics

These systems require the person to align themselves in the front of the device manually.

1.3.2 Sensor controlled lights

These systems have lights and other amenities controlled by motion sensors which are unreliable and still have manual work for custom controls.

1.3.3 Cloud-based vision

These systems use cameras to stream data to a cloud server. These are accurate but have privacy risks since video feed leaves local network and the matching data is stored on the cloud database too.

Chapter 2

Literature Review

2.1 Introduction

The development of smart office systems that combine occupancy sensing with automated personnel identification sits at the intersection of Edge AI, IoT, and computer vision. While significant research has been conducted in each of these subfields independently, their integration on resource-constrained edge hardware remains an active and relatively underexplored area. This chapter reviews the relevant literature across four key themes: the evolution of face recognition algorithms, the emergence of lightweight models for edge deployment, existing smart office and occupancy detection systems, and the tracking and threading strategies that enable real-time performance on embedded platforms.

2.2 Evolution of Face Detection and Recognition

Face detection has been a core problem in computer vision since the early 2000s. The seminal work by Viola and Jones [1] introduced the Haar Cascade classifier, a real-time object detection framework based on Haar-like features and the AdaBoost learning algorithm. Despite being over two decades old, the Haar Cascade remains widely used for its extremely low computational cost, making it ideal for detection on devices with limited processing power. This system employs the Haar Cascade classifier for exactly this reason — to maintain a high and continuous video frame rate on the Raspberry Pi while keeping the more expensive recognition step asynchronous.

The recognition side of the problem saw a paradigm shift with the introduction of deep learning-based embedding models. Schroff et al. [2] proposed FaceNet, which learns a compact Euclidean embedding space where distances directly correspond to face similarity using

a triplet loss function. FaceNet established the foundation for modern metric-learning-based face recognition and demonstrated that a single neural network could achieve state-of-the-art results by directly optimising the embedding space rather than using an intermediate classification layer.

Building on this foundation, Deng et al. [3] introduced ArcFace, which adds an additive angular margin penalty to the softmax loss to maximise the discriminative power of face embeddings. ArcFace achieves top benchmark performance on large-scale verification datasets such as LFW and IJB-C and is widely considered the current industry standard for high-accuracy face recognition. However, its high model complexity and memory footprint make direct deployment on embedded hardware challenging without significant quantisation or pruning.

2.3 Lightweight Models for Edge Deployment

The requirement to run deep learning inference on devices with constrained memory and compute — such as mobile phones and single-board computers — has driven substantial research into efficient neural architecture design. Howard et al. [4] introduced MobileNet, which replaces standard convolutions with depthwise separable convolutions, drastically reducing multiply-accumulate operations while retaining competitive accuracy. The MobileNet family established the principle that architectural efficiency, rather than post-training compression alone, is the most effective path to edge-deployable models.

In the domain of face recognition specifically, Boutros et al. [5] proposed GhostFaceNet, a face recognition model based on the GhostNet architecture, which generates feature maps using a combination of intrinsic and cheap linear operations called ghost modules. GhostFaceNet achieves recognition accuracy comparable to ArcFace on standard benchmarks while requiring significantly fewer floating point operations and less memory, making it particularly well-suited for edge deployment. This project adopts GhostFaceNet as its primary embedding model for precisely this reason — it provides a strong accuracy-to-compute trade-off that is achievable on the Raspberry Pi without requiring a GPU.

The DeepFace library, introduced by Serengil and Ozpinar [6], provides a unified Python framework wrapping multiple state-of-the-art face recognition models including FaceNet, ArcFace, and GhostFaceNet under a common API. Its modular design allows the recognition backend to be swapped without changes to the surrounding pipeline, and its support for multiple detector backends — including MTCNN for high-quality alignment — simplifies the embedding extraction process. The present system uses DeepFace for both the offline database construction phase and the online recognition phase.

2.4 Face Alignment and Multi-Task Detection

Zhang et al. [7] proposed the Multi-task Cascaded Convolutional Network (MTCNN), a cascaded architecture that jointly performs face detection and facial landmark localisation. Landmark localisation enables geometric alignment — rotating and scaling the face crop so that key landmarks such as the eyes and nose tip are consistently positioned — which substantially improves the quality of downstream embeddings. In this project, MTCNN is used during the offline database construction phase to generate high-quality, well-aligned embeddings for each enrolled individual. At runtime, however, the Haar Cascade is used for detection to reduce latency, and the detector is bypassed entirely during embedding extraction since the face is already cropped.

2.5 Object Tracking and Identity Persistence

A naive face recognition system that runs the full embedding pipeline on every detected face in every frame would be computationally intractable on edge hardware. Bewley et al. [8] addressed the analogous problem in multi-object tracking by proposing SORT (Simple Online and Realtime Tracking), which uses the Kalman filter for state prediction and the Hungarian algorithm for assignment, with Intersection-over-Union (IOU) as the association metric. The key insight of SORT is that if the overlap between a detection in the current frame and a tracked box from the previous frame is sufficiently high, they are almost certainly the same object and need not be re-identified.

This project implements a simplified variant of the IOU-based association component of SORT, without the Kalman filter. Each detected bounding box is compared against boxes tracked in the previous frame; if the IOU exceeds a threshold of 0.4, the new detection is assigned the same identity and recognition is not re-triggered. This simple mechanism is sufficient for the controlled office environment envisioned by this project and reduces the frequency of expensive embedding computations by an order of magnitude, making real-time operation on the Raspberry Pi feasible.

2.6 Smart Office and Occupancy Detection Systems

The broader context of this project is the emerging field of intelligent building automation. Existing commercially available smart office systems rely primarily on passive infrared (PIR) motion sensors for occupancy detection, which only indicate binary presence without identifying who is present. Garg et al. [9] studied energy savings from occupancy-based HVAC and lighting control and found that sensor-driven automation could reduce energy consumption by 30–40% in commercial buildings, establishing the value of accurate occupancy data. However, their systems used only coarse binary occupancy signals.

More recent work has explored cloud-based computer vision for richer occupancy and identity information. Systems such as Amazon Rekognition and Microsoft Azure Face API provide highly accurate recognition as a cloud service, but require video frames to be transmitted to remote servers, introducing latency, ongoing cost, and privacy concerns since biometric data leaves the local network. Yildirim and Curtis [10] highlighted these privacy risks in a survey of cloud-based facial recognition deployments in workplace settings and called for on-premise or edge-based alternatives. This project directly addresses that gap by building a fully local, edge-deployed recognition pipeline.

Several prior works have explored face recognition on the Raspberry Pi specifically. Rathore et al. [11] implemented an attendance system using OpenCV's LBPH (Local Binary Pattern Histogram) face recogniser on a Raspberry Pi 3B+, reporting acceptable accuracy for controlled lighting conditions but noting significant degradation in unconstrained environments. The present work advances on this by replacing the LBPH recogniser — a classical, shallow technique — with a deep embedding model (GhostFaceNet), substantially improving robustness, while still achieving real-time performance through multithreading and IOU-based tracking.

2.7 Concurrency and Real-Time Constraints on Edge Devices

A recurring challenge in edge AI systems is balancing the frame rate of the display pipeline with the latency of deep learning inference. On a GPU-equipped server, inference is fast enough that the two can be tightly coupled; on a CPU-only embedded device, this is not the case. The common architectural solution is to decouple the display and inference pipelines via a producer-consumer model implemented with threads. Bradski and Kaehler [12] discuss this pattern in the context of OpenCV-based vision systems, recommending asynchronous inference threads with shared state guarded by locks.

This project implements exactly this pattern: each new face crop is dispatched to a daemon thread that runs the GhostFaceNet embedding and cosine similarity matching independently of the main video loop. A semaphore limits the number of concurrently active recognition threads to four, preventing memory exhaustion on the Raspberry Pi when multiple faces are detected simultaneously. A threading lock protects the shared label and job-status dictionaries. This design ensures that the video feed continues to run smoothly at a stable frame rate regardless of how long individual recognition tasks take, with the displayed label updating asynchronously when a result becomes available.

2.8 Summary and Research Gap

The literature reviewed above reveals a clear trajectory: face recognition has matured from classical hand-crafted features [1] to powerful deep embedding models [2, 3], and the frontier has now shifted to making these models viable on resource-constrained hardware [4, 5].

Lightweight architectures, efficient detection pipelines, and smart tracking strategies [8] have each individually been studied. However, their combination into a single, cohesive, fully edge-deployed real-time pipeline for smart office management — with no cloud dependency [10] and a focus on practical office scenarios — represents a gap that this project addresses.

Furthermore, existing smart office solutions have not integrated personnel identification with occupancy-driven room automation at the edge [9]. This project lays the groundwork for such a system, with the face recognition engine serving as the core component that future modules — ToF-based door triggers, light and thermostat control nodes, and an automatic database enrollment interface — will build upon.

Chapter 3

Proposed Work

3.1 System overview

The proposed system is a real time face recognition system designed to operate on an edge device which in our case is a Raspberry pi. The objective is to achieve accurate and efficient real time face recognition. The system captures live video input from a camera and processes each frame to detect and identify individuals present in the scene. The design separates detection and recognition stages to allow independent optimization of each component.

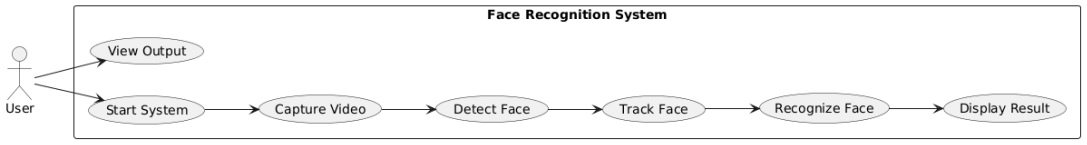


FIGURE 3.1: Use case diagram

3.2 Components used

- Raspberry Pi, 8GB RAM
- Active cooler for Raspberry Pi
- USB Camera (Logitech C922 PRO)

3.3 System architecture

The system is divided into the following modules:

- Input module: Captures video frames from the camera.

- Face detection module: Detects faces using Haar Cascade classifier.
- Tracking module: Assigns consistent IDs to detected faces using IOU-based tracking.
- Feature extraction module: Generates facial embeddings using DeepFace.
- Matching module: Compares embeddings with database using cosine similarity.
- Output module: Displays the identified name on the screen.

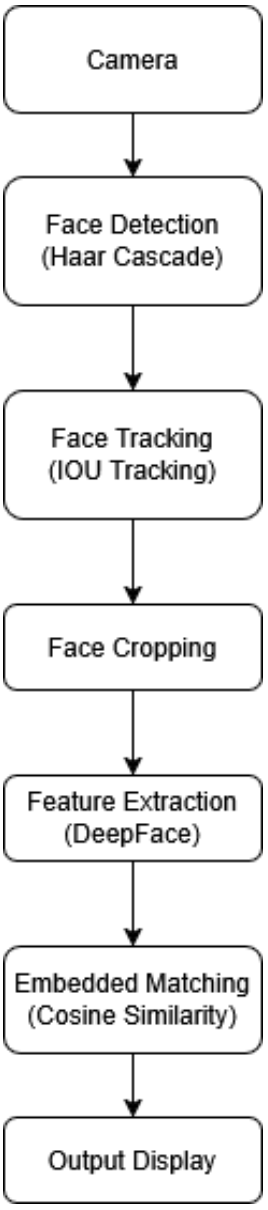


FIGURE 3.2: System Architecture of Proposed Model

3.4 System Working

The system begins by capturing frames from the camera. Each frame is converted to grayscale and passed to the face detection module where the Haar Cascade classifier detects faces and returns the bounding box coordinates.

The detected faces are then passed to the tracking module, which uses Intersection-over-Union (IOU) to assign consistent IDs to faces across consecutive frames. This prevents repeated recognition of the same face in every frame.

For each detection of a face, a cropped image is extracted and passed to the feature extraction module. The DeepFace framework is used to generate a numerical embedding vector representing the facial features.

The generated embedding is compared with precomputed database embeddings using cosine similarity. The closest match is selected, and if the similarity exceeds a predefined threshold, the corresponding identity is assigned. Otherwise, the face is labeled as unknown.

The final result is displayed on the output screen with bounding boxes and identity labels.

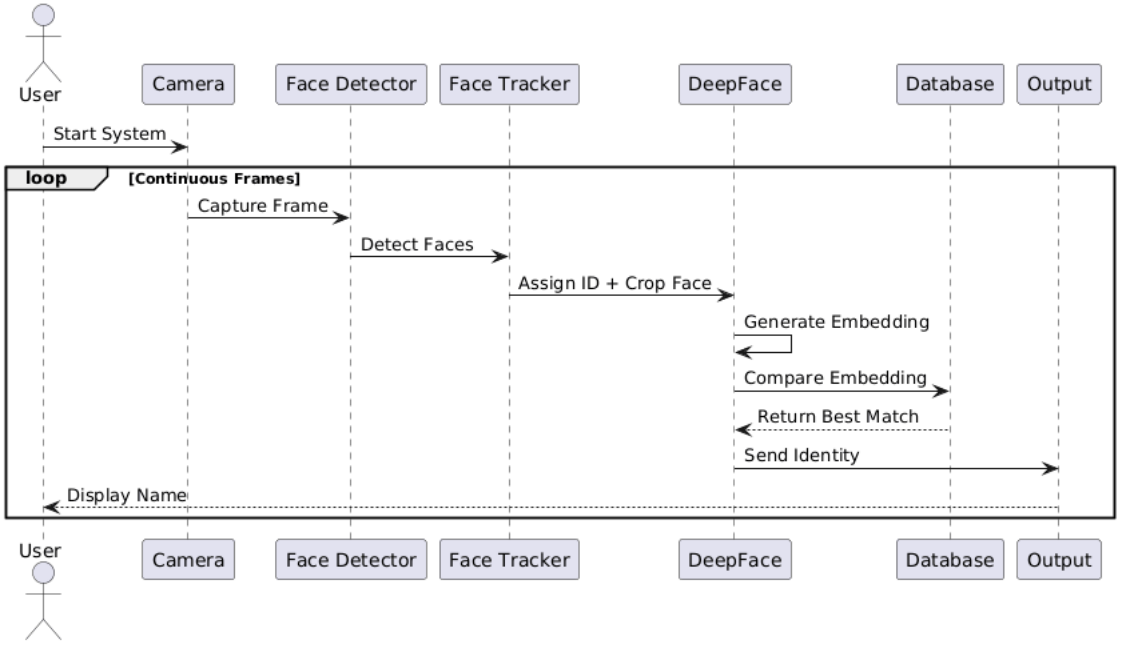


FIGURE 3.3: Sequence diagram

3.5 Database creation

The database stores the facial embeddings of known individuals. During initialization, the system scans the database directory and generates embeddings using DeepFace with MTCNN-based alignment.

Each entry in the database contains:

- Identity label
- Corresponding embedded vector

The embeddings are then stored in a serialized (.pkl) file to enable fast loading and efficient comparison during runtime.

Chapter 4

Simulation and Results

4.1 Setup

The proposed system was implemented on a Raspberry Pi with a standard USB camera. The software environment consisted of Python 3.12 along with libraries such as OpenCV, NumPy, and DeepFace.

The database consisted of face images of known individuals, organized into separate folders. During initialization, embeddings were generated and stored in a serialized format for efficient runtime comparison.

The system processes live video input and performs detection, tracking, and recognition in real-time.

4.2 Performance evaluation

4.2.0.1 Initial response time

System has an initial delay of approximately 10-12 seconds on an average during the first execution. This is caused by loading of models and initialization of embedded vectors of faces.

4.2.0.2 Real time performance

After initialization, system achieves near real time with continuous frame rate and multithreading makes sure that video feed runs smoothly while running other tasks.

4.2.0.3 Recognition Accuracy(Qualitative)

System has reliable frontal face recognition under normal lighting conditions. Face recognition works very smoothly even at a distance of 8 Feet. System is capable of handling more than one face. The use of tracking reduces redundant computation by avoiding repeated recognition of the same face across frames.

However, recognition accuracy decreases in the following circumstances:

- Poor lighting
- Partial occlusion of the face
- Extreme head angles

These are expected to be seen on constrained devices.

4.2.0.4 Stability

System maintains stable performance during continuous use due to active cooling. Usage of semaphore makes sure that system doesn't overload when multiple faces are detected.

TABLE 4.1: Performance Summary of the Proposed System

Parameter	**Observation**
Initial Delay	∼10 seconds
Runtime Performance	Real-time
Faces Supported	Multiple faces
Best Condition	Frontal face, good lighting
Weakness	Side angles, poor lighting

4.3 Limitations

Limitations of system are:

- Reduced accuracy while sudden and vigorous facial movements.
- Initial delay during model loading.
- Limited performance compared to GPU based systems.

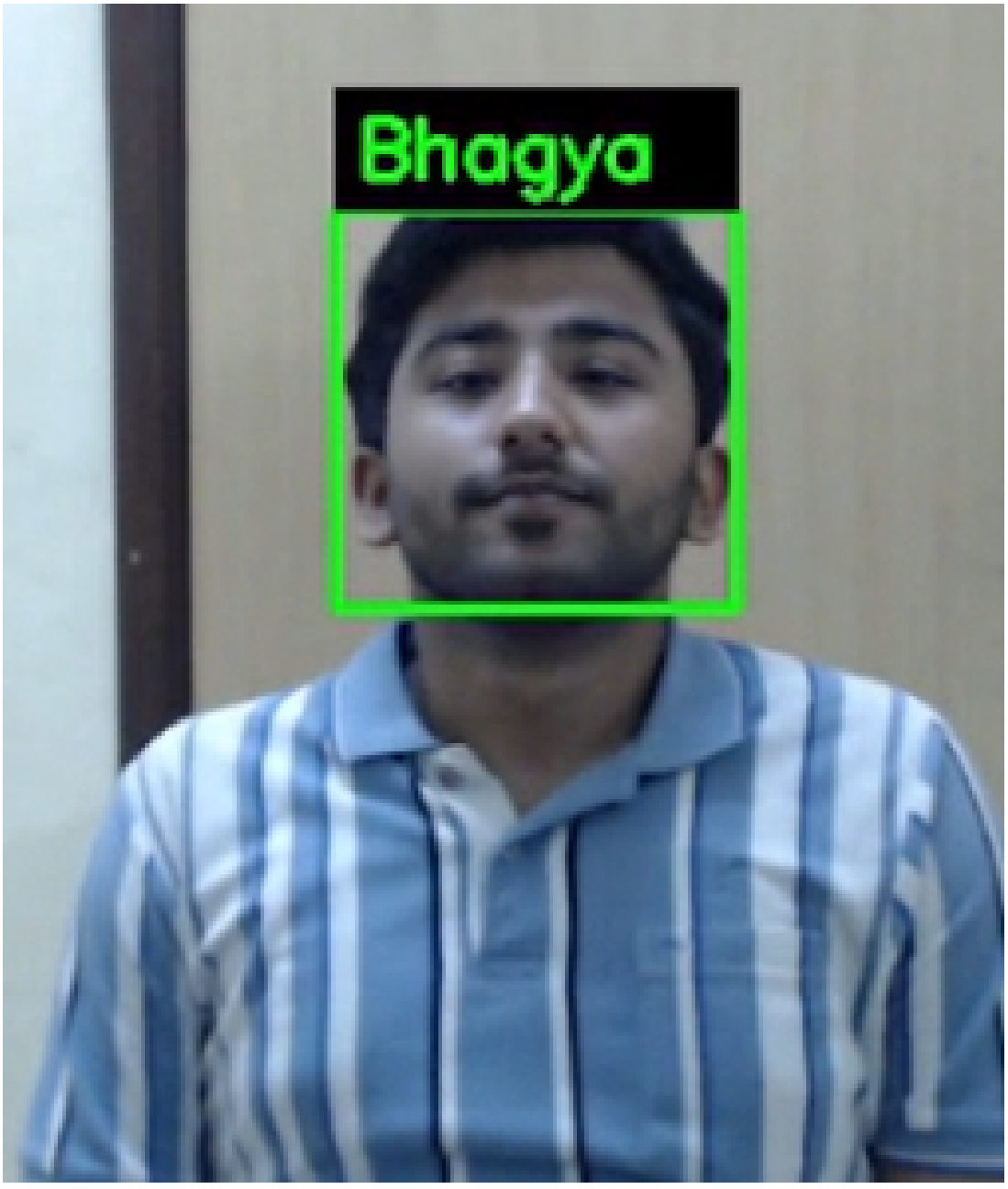


FIGURE 4.1: Single face

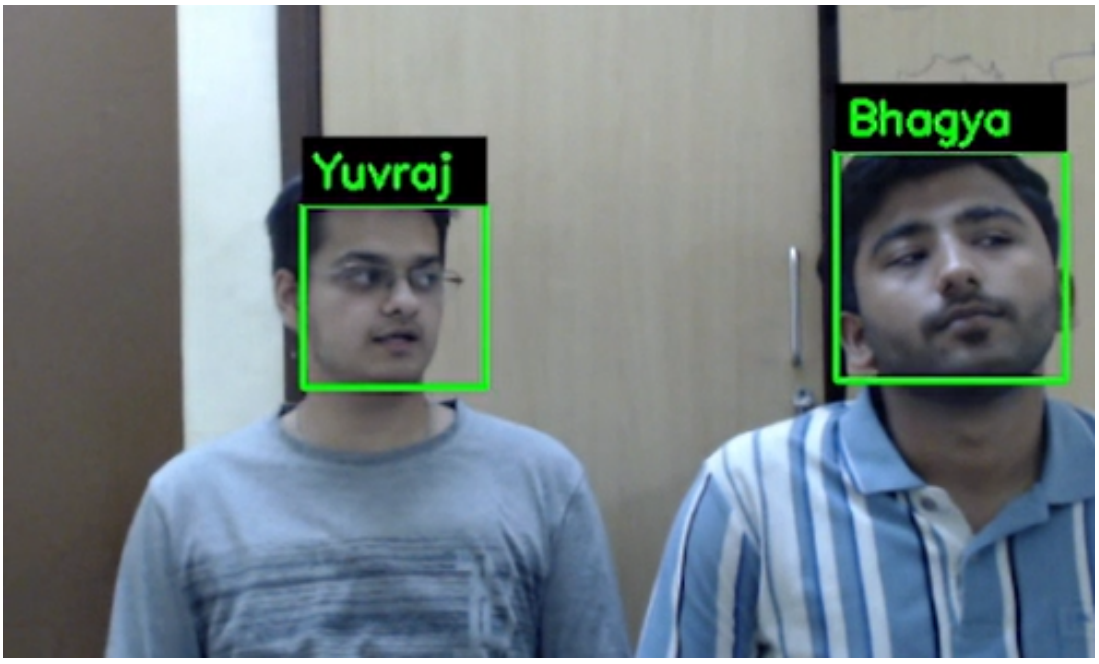


FIGURE 4.2: Multiple Faces

Chapter 5

Conclusions and Future Work

This proposed system demonstrates that real-time face recognition can be effectively implemented on edge devices using an optimized modular pipeline. By combining efficient detection, tracking, and embedding-based recognition, the system achieves a balance between computational efficiency and accuracy, making it suitable for smart office applications. Now, to make this prototype into a full fledged smart office, the following work will be done:

- An ESP32-C3 node will be integrated with a Time-of-Flight (ToF) sensor. This node will act as a low-power interrupt, only activating the Raspberry Pi vision pipeline when the door is opened. This will save a lot of power
- The server will have the feature to capture unknown people and user can label them and add to database
- Another node which controls room lights and thermostat will be connected to server which can change the lighting and temperature based on occupants in the room.
- Developing a meeting mode which can adjust some factors accordingly.

Bibliography

[1] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 511–518, 2001.

[2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 815–823, 2015.

[3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4690–4699, 2019.

[4] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[5] F. Boutros, N. Damer, F. Kirchbuchner, and A. Kuijper, "GhostFaceNets: Lightweight face recognition model from cheap operations," *IEEE Access*, vol. 11, pp. 35429–35446, 2023.

[6] S. I. Serengil and A. Ozpinar, "LightFace: A hybrid deep face recognition framework," in *Proceedings of the IEEE International Conference on Innovations in Intelligent Systems and Applications (ASYU)*, 2020.

[7] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.

[8] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pp. 3464–3468, 2016.

[9] V. Garg, N. Bansal, and S. Jain, "Lighting energy savings opportunities in office buildings," in *Proceedings of the ACEEE Summer Study on Energy Efficiency in Buildings*, 2000.

[10] K. Yildirim and L. Curtis, "Privacy implications of facial recognition in workplace environments," *Journal of Information Privacy and Security*, vol. 17, no. 3, 2021.

[11] S. Rathore, A. Sharma, and R. Gupta, "Raspberry Pi based real-time face recognition attendance system," in *Proceedings of the IEEE International Conference on Innovative Trends in Information Technology (ICIIT)*, 2021.

[12] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.